

Mục tiêu

- Nắm được các kiến thức cơ bản về con trỏ (pointer).
- Quản lý bộ nhớ với con trỏ (pointer).

Yêu cầu

- Bài A: hoàn thành 1 bài bất kỳ trong bài tập.
- Bài H: hoàn thành đủ 6 bài.
- Mỗi bài làm trong 1 project riêng, tất cả các bài làm trong 1 solution.

Bài tập

Bài 1

Thực hiện các yêu cầu sau với con trỏ:

- Viết hàm phát sinh mảng (số lượng phần tử ít nhất là 3, giá trị mỗi phần tử nằm trong khoảng cho trước).
- Tìm phần tử lớn nhất và nhỏ nhất của mảng.
- Tính tổng tất cả các phần tử của mảng.
- Tính tổng tất cả các phần tử có số lần xuất hiện lớn nhất.
- Tìm phần tử trung vị của mảng.
- Tìm mảng con của mảng (xác định 1 mảng có là con của mảng đã có hay không).

Bài 2

Viết chương trình phát sinh mảng với con trỏ (số lượng phần tử ít nhất là 10, giá trị mỗi phần tử nằm trong khoảng $[-1000, 2000]$). Thực hiện các yêu cầu sau:

- Tìm số chính phương đầu tiên trong mảng.
- Tìm mảng con là tất cả các số nguyên tố trong mảng.

Bài 3

Viết chương trình phát sinh 2 mảng động với con trỏ p, q (số lượng phần tử ít nhất 10, giá trị mỗi phần tử nằm trong khoảng $[-50, 50]$). Sau đó thực hiện các yêu cầu sau:

- Xuất ra màn hình các phần tử chỉ có trong mảng p mà không có trong mảng q.
- Tạo 1 mảng là hợp của 2 mảng p, q sao cho các phần tử khác nhau đôi một.

Bài 4

Viết chương trình sử dụng con trỏ cấu trúc để hiển thị giờ, phút, giây ra màn hình, và tính khoảng cách giữa 2 mốc thời gian. Áp dụng cho hàm main sau:

```
struct ThoiGian
{
    int gio, phut, giay;
};
```

```

ThoiGian* TaoThoiGian(int h, int m, int s)
{
    // ...
}

void HopLeThoiGian(ThoiGian* pTG)
{
    // ...
}

void XuatTG(ostream &os, const ThoiGian* pTG)
{
    // ...
}

ThoiGian* KhoangCachThoiGian(ThoiGian* pTG1, ThoiGian* pTG2)
{
    // ...
}

void main1()
{
    // khởi động random
    srand(3072);
    // khai báo con trỏ ThoiGian
    ThoiGian *pTG1, *pTG2;
    // tạo giá trị
    pTG1 = TaoThoiGian(rand() % 100, rand() % 100, rand() % 100);
    pTG2 = TaoThoiGian(rand() % 100, rand() % 100, rand() % 100);
    // xuất ra màn hình
    cout << "Thoi gian 1: ";
    XuatTG(cout, pTG1);
    cout << endl << "Thoi gian 2: ";
    XuatTG(cout, pTG2);
    cout << endl << endl;
    // làm hợp lệ thời gian
    HopLeThoiGian(pTG1);
    HopLeThoiGian(pTG2);
    cout << "Thoi gian sau khi hop le";
    cout << endl << "Thoi gian 1: ";
    XuatTG(cout, pTG1);
    cout << endl << "Thoi gian 2: ";
    XuatTG(cout, pTG2);
    cout << endl << endl;
    // tính khoảng cách
    ThoiGian* p = KhoangCachThoiGian(pTG1, pTG2);
    cout << "Khoang cach 2 thoi gian tren: ";
    XuatTG(cout, p);
    cout << endl;

    system("pause");
}

```

Kết quả

```

Thoi gian 1: 73:47:20
Thoi gian 2: 87:71:28

Thoi gian sau khi hop le
Thoi gian 1: 73:48:10
Thoi gian 2: 88:11:28

Khoang cach 2 thoi gian tren: 14:23:18

```

Bài 5

Viết chương trình sử dụng con trỏ cấu trúc thể hiện ngày, tháng, năm ra màn hình, và tính khoảng cách giữa 2 ngày.

Bài 6

Xây dựng struct DuongTron như yêu cầu và hàm main sau chạy đúng:

```
#define PI 3.14159

struct Diem
{
    float x, y;
};

struct DuongTron
{
    Diem* tam;
    float banKinh;
};

DuongTron* TaoDuongTron(float xV, float yV, float rV);
void XoaDuongTron(DuongTron* p);
void PrintDuongTron(ostream &os, DuongTron* p);
DuongTron* ChuViLonNhat(DuongTron** l, int n);
float TongDienTich(DuongTron** l, int n);
int XuatCacDuongTronDTLonHonTB(DuongTron** l, int n);
void XuatDuongTronChuViGiam(DuongTron** l, int n);
void XoaDsDuongTron(DuongTron** l, int n);

void main2()
{
    srand(4003);

    // số lượng đường tròn
    int n = 5 + rand() % 20;

    // tạo mảng các đường tròn
    DuongTron* *listDT = new DuongTron*[n];
    if (listDT == NULL)
    {
        return;
    }
    float xV, yV, rV;
    for (int i = 0; i < n; ++i)
    {
        xV = (rand() % 1000) * 1.0f / 50;
        yV = (rand() % 1000) * 1.0f / 50;
        rV = (rand() % 1000 + 1) * 1.0f / 50;
        listDT[i] = TaoDuongTron(xV, yV, rV);
    }

    // Xuất đường tròn có chu vi lớn nhất
    cout << "Duong tron co chu vi lon nhat: ";
    DuongTron* pDTPMax = ChuViLonNhat(listDT, n);
    PrintDuongTron(cout, pDTPMax);
    cout << endl << endl;

    // Tổng diện tích các đường tròn
    cout << "Tong dien tich: " << TongDienTich(listDT, n);
    cout << endl << endl;

    // Xuất các đường tròn có diện tích lớn hơn diện tích trung bình
    cout << "Cac duong tron co dien tich lon hon TB:";
    int dem = XuatCacDuongTronDTLonHonTB(listDT, n);
```

```

cout << "Số lượng: " << dem;
cout << endl << endl;

// Xuất ds đường tròn giảm dần theo chu vi
cout << "Ds duong tron chu vi giam dan: ";
XuatDuongTronChuViGiamDan(listDT, n);
cout << endl << endl;

// thu hồi bộ nhớ
XoaDsDuongTron(listDT, n);

system("pause");
}

```

Kết quả:

```

Duong tron co chu vi lon nhat: [(3.34,18.86), 14.80]
Tong dien tich: 3977.41

Cac duong tron co dien tich lon hon TB:
S trung binh: 265.16
[(4.12,14.80), 14.62] voi S=671.49
[(8.32,17.14), 13.34] voi S=559.06
[(11.60,5.80), 13.32] voi S=557.38
[(1.92,14.30), 13.50] voi S=572.55
[(3.34,18.86), 14.80] voi S=688.13
[(5.88,4.42), 14.18] voi S=631.68
Số lượng: 6

Ds duong tron chu vi giam dan:
[(3.34,18.86), 14.80] voi P=92.99
[(4.12,14.80), 14.62] voi P=91.86
[(5.88,4.42), 14.18] voi P=89.09
[(1.92,14.30), 13.50] voi P=84.82
[(8.32,17.14), 13.34] voi P=83.82
[(11.60,5.80), 13.32] voi P=83.69
[(12.18,16.22), 7.82] voi P=49.13
[(1.10,17.88), 3.80] voi P=23.88
[(16.52,12.12), 2.80] voi P=17.59
[(0.00,7.96), 2.08] voi P=13.07
[(8.06,13.76), 2.06] voi P=12.94
[(12.74,17.48), 1.06] voi P=6.66
[(9.92,3.26), 1.04] voi P=6.53
[(14.04,17.66), 0.60] voi P=3.77
[(18.56,0.72), 0.10] voi P=0.63

```

Nội dung

Con trỏ

Cần hiểu rõ: giá trị của biến con trỏ là địa chỉ vùng nhớ của biến khác hoặc vùng nhớ hợp lệ đã được cấp phát.

Cách khai báo:

```

/// khai báo biến con trỏ kiểu int
int *p;
int a = 10;

/// gán giá trị biến con trỏ là địa chỉ biến khác
p = &a;

/// cấp phát vùng nhớ riêng và sử dụng
p = (int*)malloc(sizeof(int));

```

```

/// hoặc sử dụng toán tử new
p = new int;

if (p != NULL)
{
    *p = 100;
}

```

Phép toán đối với con trỏ: chủ yếu phép + và -, trong đó về bản chất là thay đổi địa chỉ lưu bởi con trỏ tương ứng với kiểu dữ liệu (nghĩa là dịch chuyển địa chỉ đi 1 khoảng tương ứng với số byte của kiểu dữ liệu).

Xét ví dụ sau:

```

//khai báo 2 con trỏ p kiểu int (4 byte) và q kiểu char (1 byte)
int *p;
char *q;
int a = 10;

//gán giá trị cho cả 2 con trỏ là địa chỉ của biến a
p = &a;
q = (char*)&a;

//vì cùng giá trị địa chỉ (là biến a) nên hiệu bằng 0
int hieu = (int)p - (int)q;

//khi cộng thêm 1 đơn vị thì hiệu không là 0 mà là 3
//lý do: kiểu int 4 byte nên p + 1 sẽ tăng giá trị địa chỉ lên 4,
//còn kiểu char chỉ 1 byte nên q + 1 chỉ tăng giá trị địa chỉ lên 1
hieu = (int)(p + 1) - (int)(q + 1);

```

Sử dụng con trỏ với mảng tĩnh: dựa vào ý nghĩa phép toán của con trỏ ở trên ứng dụng con trỏ để duyệt mảng.

```

//khai báo mảng tĩnh
int mang[10];
//khai báo con trỏ lưu giá trị là địa chỉ đầu mảng
int *p = mang;
//duyet mang bình thường dựa vào con trỏ để gán giá trị mảng về 0
for (int i=0; i<10; ++i)
{
    *(p+i) = 0;
    //tương đương a[i] = 0
}

```

Sử dụng con trỏ với mảng cấp phát động: không như mảng tĩnh, phải khai báo số lượng phần tử tối đa ngay từ đầu, với con trỏ thì cho phép xin cấp phát đúng số lượng vùng nhớ cần thiết.

```

/// khai báo con trỏ là mảng động
int n = 10;
int *p;

/// sử dụng malloc
p = (int*)malloc(sizeof(int) * n);
/// hoặc sử dụng calloc
p = (int*)calloc(n, sizeof(int));
/// hoặc sử dụng toán tử new
p = new int[n];

```



```

if (p == NULL)
{
    return;
}
/// sau đó sử dụng như mảng thường
for (int i = 0; i < n; ++i)
{
    /// 2 dòng code là tương đương
    *(p + i) = 0;
    p[i] = 0;
}
/// sau khi sử dụng con trỏ thì phải nhớ thu hồi vùng nhớ
free(p);    /// nếu sử dụng hàm cấp phát malloc/calloc
delete[] p;  /// nếu sử dụng toán tử new

```

Kỹ thuật phát sinh số ngẫu nhiên

Để phát sinh số ngẫu nhiên cần sử dụng 2 thư viện <time.h> (để lấy số thời gian hiện tại) và thư viện <stdlib.h> (chứa hàm phát sinh số ngẫu nhiên).

Trước khi sử dụng được hàm phát sinh số ngẫu nhiên rand() thì trước hết phải thực hiện gieo hạt giống cho nó thông qua hàm srand(unsigned int), và chỉ cần gieo 1 lần đầu cho suốt chương trình là được.

Vấn đề cần có giá trị trong khoảng [a,b] mong muốn thì sử dụng phép đồng dư (%).

```

//cần có dòng lệnh này trước khi gọi hàm rand() lần đầu tiên
srand((unsigned)time(NULL));
//giá trị lấy được sẽ nằm trong khoảng [a, b]
int x = rand() % (b - a + 1) + a;

```

Con trỏ và struct

Như đã học ở tuần trước thì struct thực ra là kiểu dữ liệu do người dùng tự định nghĩa. Vì vậy sử dụng con trỏ với struct là hoàn toàn tương tự kiểu dữ liệu nguyên thủy (int, long, double,...)

Ví dụ cơ bản

```

#include<iostream>
using namespace std;

struct PhanSo
{
    int tu, mau;
};

int main()
{
    PhanSo* pPS = new PhanSo;
    if (pPS == NULL)
    {
        return -1;
    }
    pPS->tu = 9;
    pPS->mau = 10;
    cout << pPS->tu << "/" << pPS->mau << endl;
    delete pPS;

    /// giữ console
    system("pause");
    return 0;
}

```

Với con trỏ thì luôn phải nhớ cấp phát trước khi sử dụng và thu hồi khi không dùng nữa.

Con trỏ trong truyền tham số

Vì giá trị của biến con trỏ là địa chỉ vùng nhớ nên khi truyền tham số là con trỏ thì nghĩa là truyền địa chỉ vùng nhớ (tương tự truyền tham số kiểu tham chiếu với toán tử &).

Xét ví dụ sau:

```
#include<iostream>
using namespace std;

struct PhanSo
{
    int tu, mau;
};

void XuatPhanSo(ostream& os, const PhanSo &pPS)
{
    os << pPS.tu << "/" << pPS.mau;
}

void DaoPhanSo(PhanSo ps)
{
    int temp = ps.mau;
    ps.mau = ps.tu;
    ps.tu = temp;
}

void DaoPhanSoPointer(PhanSo *pPS)
{
    int temp = pPS->mau;
    pPS->mau = pPS->tu;
    pPS->tu = temp;
}

int main()
{
    PhanSo* pPS = new PhanSo{ 9, 10 };
    if (pPS == NULL)
    {
        return -1;
    }
    cout << "Sau khi chạy DaoPhanSo(PhanSo): ";
    DaoPhanSo(*pPS);
    XuatPhanSo(cout, *pPS);
    cout << endl << "Sau khi chạy DaoPhanSoPointer(PhanSo*): ";
    DaoPhanSoPointer(pPS);
    XuatPhanSo(cout, *pPS);
    cout << endl;
    delete pPS;

    /// giữ console
    system("pause");
    return 0;
}
```

Lưu ý toán tử * để lấy giá trị tại địa chỉ con trỏ lưu giữ (sử dụng khi cần truyền tham số kiểu tham trị đối với con trỏ như ví dụ trên).

Hướng dẫn một vài câu cho Bài 1

```
/// hàm phát sinh mảng
int* PhatSinhMang(const int &min, const int &max)
```

```

{
    /// phát sinh số phần tử
    int n = 3 + rand();
    /// cấp phát mảng (chứa chỗ lưu trữ cả n)
    int *arr = new int[n + 1];
    /// giữ giá trị n ở ô đầu tiên
    arr[0] = n;
    /// phát sinh giá trị phần tử
    int d = max - min + 1;
    for (int i = 0; i < n; i++)
    {
        arr[i + 1] = min + rand() % d;
    }
    /// trả về pointer trở từ vị trí phần tử
    return arr + 1;
}

/// hàm tìm phần tử lớn nhất và nhỏ nhất
void TimMinMax(const int *arr, int &minE, int &maxE)
{
    /// lấy lại giá trị n
    int n = *(arr - 1);
    /// tìm phần tử min và max
    minE = maxE = *arr; // hoặc = arr[0]
    for (int i = 1; i < n; i++)
    {
        if (arr[i] > maxE)
        {
            maxE = arr[i];
        }
        if (arr[i] < minE)
        {
            minE = arr[i];
        }
    }
}

/// hàm tính tổng tất cả các phần tử có số lần xuất hiện nhiều nhất
int TongXuatHienMax(const int *arr)
{
    int n = *(arr - 1);
    /// tạo 2 mảng để đếm số lần xuất hiện
    int *arrGiaTri = new int[n];
    int *arrDem = new int[n];
    /// thực hiện đếm
    int iDem = 0, nDem = 0, maxDem = 0;
    for (int i = 0; i < n; i++)
    {
        /// với mỗi giá trị sẽ kiểm tra xem đã từng xuất hiện chưa
        iDem = 0;
        for (; iDem < nDem; iDem++)
        {
            if (arrGiaTri[iDem] == arr[i])
            {
                break;
            }
        }
        if (iDem == nDem)
        {
            /// có nghĩa chưa từng xuất hiện
            arrGiaTri[iDem] = arr[i];
            arrDem[iDem] = 0;
            nDem++;
        }
        arrDem[iDem]++;
        if (maxDem < arrDem[iDem])
        {
            maxDem = arrDem[iDem];
        }
    }
}

```



```
    }  
}  
/// tính tổng  
int total = 0;  
for (int i = 0; i < n; i++)  
{  
    iDem = 0;  
    for (; iDem < nDem; iDem++)  
    {  
        if (arrGiaTri[iDem] == arr[i])  
        {  
            break;  
        }  
    }  
    if (arrDem[iDem] == maxDem)  
    {  
        total += arr[i];  
    }  
}  
/// cần delete arrGiaTri và arrDem  
return total;  
}
```